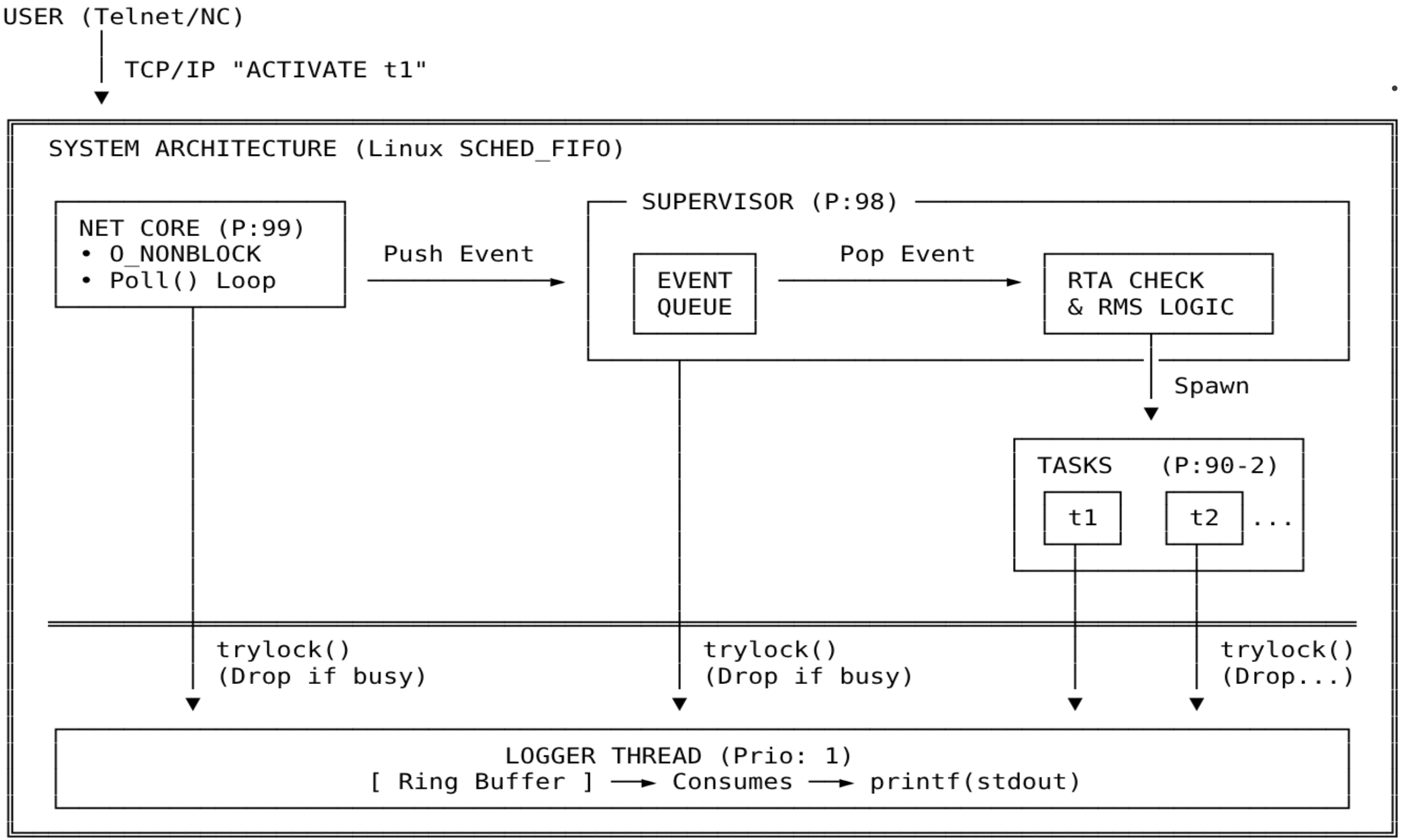
Dynamic Periodic Task

Architecture



Network Core

01

Mechanism

I/O Multiplexing via poll() on a single thread

02

Configuration

Sockets set to O_NONBLOCK, so one thread serves more clients

03

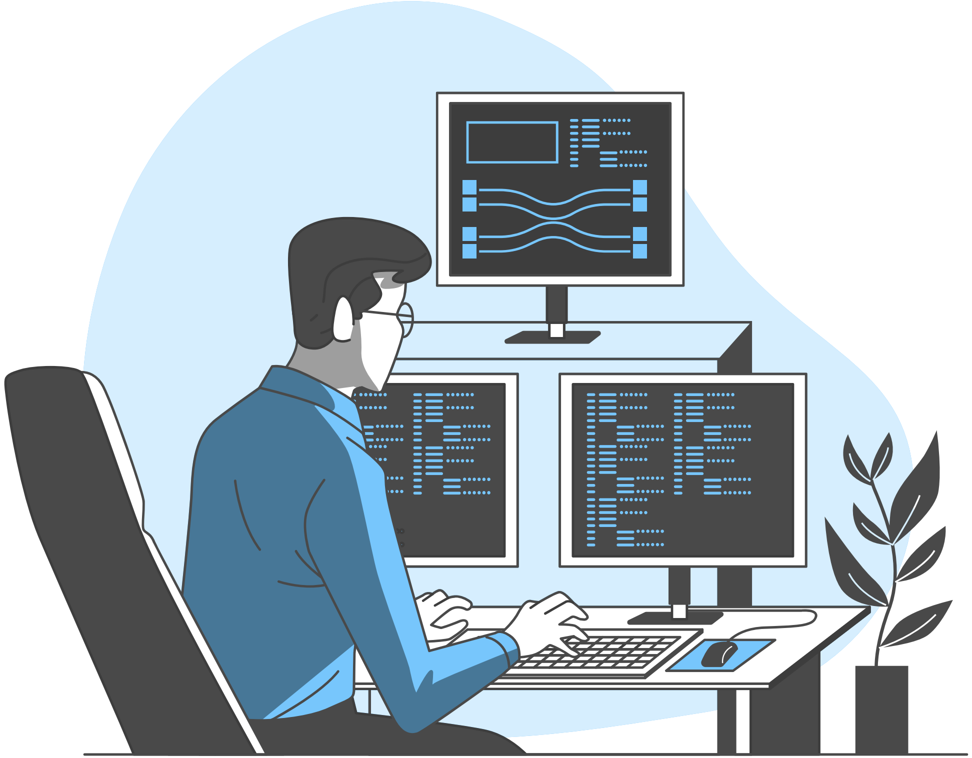
Maximum Priority

Drain sockets fast, avoid client drops

04

Data Integrity

Ring buffers to handle TCP fragmentation



Real-Time Scheduling & RTA

Policy

Rate Monotonic Scheduling (RMS): optimal fixed-priority policy where shorter periods receive higher priority.

Admission Control

New tasks are reject if their worst-case response time, under higher-priority interference, exceeds deadline.

Timing Loop

Uses clock_nanosleep with TIMER_ABSTIME to ensure zero accumulated drift

...

Measures execution time to detect deadline misses and task lateness

What if a deadline is missed?

The Problem

If a task is delayed by seconds, its next activation is in the past.

⇩

Task loops immediately to "catch up", starving lower-priority tasks.

The Solution

If next_activation <= now, jump forward by N periods.

⇩

Maintains the original time grid alignment and recovers system stability instantly.

Why a custom Logger?

Problem

The printf() sys-call is thread-safe by using internal locking

⇩

if a lower-priority thread holds this lock, a higher-priority thread can block behind it, adding unbounded latency

Solution

Dedicate a low-priority logger thread that does the blocking printf() work.

⇩

real-time threads call pthread_mutex_trylock(); if contended, the log message is dropped (never block the producer).

DEMO